

Formal RL Length Generalization

A technical handbook for the 'formal_rl_length_generalization' implementation.

Date: June 4, 2026

Preface

This handbook explains the theory and implementation behind the 'formal_rl_length_generalization' project. The project studies whether small autoregressive Transformer policies can learn length-generalizing algorithms for formal-language recognition when trained with PPO or GRPO and oracle process rewards.

The core experimental question is whether dense rewards on intermediate reasoning states help a model learn an algorithm, rather than merely fitting short training examples.

The codebase implements four task families:

- parity
- modular counting with 'k = 3' and 'k = 5'
- ' $a * b$ '
- ' $a^n b^n$ '

It supports seven training regimes:

- supervised CoT baseline
- PPO terminal-only reward
- PPO oracle process reward
- PPO process plus terminal reward
- GRPO terminal-only reward
- GRPO oracle process reward
- GRPO process plus terminal reward

Training lengths are restricted to:

```
"""text
n = 1, 2, ..., 40
"""
```

OOD evaluation lengths are:

```
"""text
41-80
81-160
161-320
```

““

The intended scientific discipline is important: do not train or tune on OOD lengths.

1. Research Question

The main question is:

““text

Can small Transformer policies trained with PPO/GRPO and oracle process rewards learn length-generalizing algorithms for formal languages better than terminal-only RL?

““

The hypothesis is:

““text

process reward -> better state tracking -> more algorithmic policy -> better length generalization

““

The point is not merely to obtain high training accuracy. The point is to see whether the learned policy extrapolates from short inputs to much longer ones.

2. General Problem Setup

Each task defines an input sequence:

““text

$x = (x_1, x_2, \dots, x_n)$

““

an oracle intermediate state trace:

““text

$s = (s_1, s_2, \dots, s_n)$

““

and a final answer:

““text

y_{final}

““

The model receives a prompt:

```
"""text
TASK <task_name> INPUT x_1 x_2 ... x_n TRACE
"""
```

and must generate:

```
"""text
s_1 s_2 ... s_n FINAL y_final
"""
```

For ' $a^n b^n$ ', the trace contains two tokens per timestep:

```
"""text
phase_t BAL_balance_t
"""
```

so the target is:

```
"""text
phase_1 BAL_b1 phase_2 BAL_b2 ... phase_n BAL_bn FINAL accept/reject
"""
```

The model is therefore trained not only to classify a string, but also to emit an explicit computation trace.

3. Reward Decomposition

All RL algorithms use two basic reward components.

The process reward measures correctness of the intermediate trace:

```
"""text
R_process = average intermediate-state correctness
"""
```

The terminal reward measures correctness of the final answer:

```
"""text
R_terminal = 1[predicted final answer is correct]
"""
```

The combined reward mode is:

```
"""text
R = lambda_p R_process + lambda_T R_terminal
"""
```

where 'lambda_p' is 'process_weight' and 'lambda_T' is 'terminal_weight'.

The three RL reward modes are:

```
"""text
terminal-only:      R = R_terminal
process-only:       R = R_process
process-plus-terminal: R = lambda_p R_process + lambda_T R_terminal
"""
```

4. Task 1: Parity

The parity language is the set of binary strings with even parity.

Input symbols:

```
"""text
x_t in {0, 1}
"""
```

The hidden state is a parity bit:

```
"""text
p_0 = 0
p_t = p_{t-1} XOR x_t
"""
```

The trace is:

```
"""text
p_1 p_2 ... p_n
"""
```

The final answer is:

```
"""text
p_n
"""
```

In the implementation, states are emitted as 'P0' and 'P1'.

The process reward is:

```

"""text
R_process = (1/n) sum_{t=1}^n 1[hat_p_t = p_t]
"""

```

The terminal reward is:

```

"""text
R_terminal = 1[hat_p_final = p_n]
"""

```

Parity is a regular language and should be solvable with one bit of memory. It is a sanity-check task for algorithmic state tracking.

5. Task 2: Modular Counting

The modular counting task asks whether the number of '1' symbols is divisible by 'k'.

The implementation supports:

```

"""text
k = 3
k = 5
"""

```

The state is:

```

"""text
c_0 = 0
c_t = (c_{t-1} + x_t) mod k
"""

```

The final accept condition is:

```

"""text
accept iff c_n = 0
"""

```

The process reward is:

```

"""text
R_process = (1/n) sum_{t=1}^n 1[hat_c_t = c_t]
"""

```

The terminal reward is:

```

"""text
R_terminal = 1[hat_accept = accept]
"""

```

This is also regular, but it requires a cyclic state machine with 'k' states.

6. Task 3: a^*b^*

The language ' a^*b^* ' contains strings with zero or more 'a's followed by zero or more 'b's. Once 'b' appears, no 'a' may appear afterward.

The DFA states are:

```

"""text
S0 = still reading a's
S1 = reading b's
S2 = dead/invalid
"""

```

The transition function is:

```

"""text
delta(S0, a) = S0
delta(S0, b) = S1
delta(S1, b) = S1
delta(S1, a) = S2
delta(S2, a) = S2
delta(S2, b) = S2
"""

```

The accept condition is:

```

"""text
accept iff  $s_n$  in {S0, S1}
"""

```

The process reward is:

```

"""text
R_process =  $(1/n) \sum_{t=1}^n 1[\hat{s}_t = s_t]$ 
"""

```

The terminal reward is:

```

"""text
R_terminal = 1[hat_accept = accept]
"""

```

““

This task tests phase tracking and invalid-state detection.

7. Task 4: $a^n b^n$

The language ' $a^n b^n$ ' is context-free. A valid string has a block of 'a's followed by an equally long block of 'b's.

The implementation uses a canonical state with two components:

```
““text
phase_t in {A_PHASE, B_PHASE, DEAD}
balance_t = count_a_t - count_b_t
““
```

Counts are:

```
““text
count_a_t = number of a's seen up to time t
count_b_t = number of b's seen up to time t
““
```

The balance is:

```
““text
balance_t = count_a_t - count_b_t
““
```

The phase tracks whether the sequence is still in the 'a' block, has entered the 'b' block, or has become invalid.

The model emits:

```
““text
phase_t BAL_balance_t
““
```

at every timestep.

The phase reward is:

```
““text
R_phase(t) = 1[hat_phase_t = phase_t]
““
```

The balance reward is shaped:

```
"""text
R_balance(t) = 1 - min(abs(hat_balance_t - balance_t), cap) / cap
"""
```

with:

```
"""text
cap = 500
"""
```

The per-step reward is:

```
"""text
r_t = 0.5 R_phase(t) + 0.5 R_balance(t)
"""
```

The process reward is:

```
"""text
R_process = (1/n) sum_{t=1}^n r_t
"""
```

The terminal reward is:

```
"""text
R_terminal = 1[hat_accept = accept]
"""
```

This is the hardest task because the correct solution resembles a counter or stack-like computation.

8. Tokenization

The tokenizer is a fixed symbolic tokenizer. There is no BPE, no learned tokenizer, and no subword splitting.

Special tokens:

```
"""text
<pad>
<bos>
<eos>
TASK
INPUT
"""
```


TRACE

FINAL

""

Task and state tokens include:

""text

0, 1, a, b

P0, P1

C0, C1, ...

S0, S1, S2

A_PHASE, B_PHASE, DEAD

BAL_-500 ... BAL_500

ACCEPT, REJECT

""

This makes the experiment cleaner: the model is not spending capacity discovering a representation for symbols. It is learning the transition behavior over already-symbolic tokens.

Encoding:

""text

token -> integer ID

""

Decoding:

""text

integer ID -> token

""

Batches are padded with '<pad>'. Prompt positions are masked out of the supervised loss and RL action loss.

9. Transformer Architecture

The model is a causal Transformer policy.

For input token IDs:

""text

z_1, z_2, ..., z_T

""

the model forms:

```

"""text

$$h_t^0 = E_{\text{token}}(z_t) + E_{\text{pos}}(t)$$

"""

```

where 'E_token' is the token embedding and 'E_pos' is the positional embedding.

The causal attention mask is:

```

"""text

$$M_{ij} = 0 \quad \text{if } j \leq i$$


$$M_{ij} = -\text{infinity} \quad \text{if } j > i$$

"""

```

So position 'i' can only attend to positions up to 'i'.

After the Transformer stack, the final hidden state is:

```

"""text

$$h_t$$

"""

```

The policy head produces logits:

```

"""text

$$\text{logits}_t = W_{\pi} h_t + b_{\pi}$$

"""

```

The token distribution is:

```

"""text

$$\pi_{\theta}(a_t | z_{\leq t}) = \text{softmax}(\text{logits}_t)$$

"""

```

The value head produces:

```

"""text

$$V_{\theta}(z_{\leq t}) = W_V h_t + b_V$$

"""

```

The policy head is used by SFT, PPO, and GRPO. The value head is used by PPO.

10. Autoregressive Generation

Given prompt 'q', the model samples:

```

"""text
a_t ~ pi_theta(. | q, a_1, ..., a_{t-1})
"""

```

The full sampled completion probability is:

```

"""text
pi_theta(a_1:m | q)
= product_{t=1}^m pi_theta(a_t | q, a_{<t})
"""

```

The log probability is:

```

"""text
log pi_theta(a_1:m | q)
= sum_{t=1}^m log pi_theta(a_t | q, a_{<t})
"""

```

Generation stops when '<eos>' is sampled or when 'max_new_tokens' is reached.

11. Supervised CoT Baseline

The supervised baseline trains the model with oracle traces by teacher forcing.

Prompt:

```

"""text
q = TASK task INPUT x_1 ... x_n TRACE
"""

```

Target:

```

"""text
y = s_1 s_2 ... s_n FINAL y_final <eos>
"""

```

The SFT loss is cross-entropy over target positions only:

```

"""text
L_SFT(theta)
= -(1/M) sum_{t=1}^M log pi_theta(y_t | q, y_{<t})
"""

```

Prompt tokens do not contribute to the loss.

The token accuracy metric is:

```

"""text
Acc_token
= (1/M) sum_{t=1}^M 1[argmax_a pi_theta(a | q, y_{<t}) = y_t]
"""

```

SFT directly teaches the oracle computation trace. The RL variants instead reward traces after the policy samples them.

12. Reinforcement Learning View

In RL, the model is a policy:

```

"""text
pi_theta(a_t | s_t)
"""

```

where the RL state is the autoregressive prefix:

```

"""text
s_t = (q, a_{<t})
"""

```

A rollout is:

```

"""text
tau = (q, a_1, ..., a_m)
"""

```

The objective is:

```

"""text
J(theta) = E_{tau ~ pi_theta}[R(tau)]
"""

```

The policy-gradient identity is:

```

"""text
grad_theta J(theta)
= E[sum_t grad_theta log pi_theta(a_t | s_t) A_t]
"""

```

The main question is how to define the advantage 'A_t'.

PPO uses:

```

"""text
A_t = R_t - V_theta(s_t)
"""

```

GRPO uses group-relative reward:

```

"""text
A_i,k = (R_i,k - mean_k R_i,k) / (std_k R_i,k + epsilon)
"""

```

13. PPO

PPO compares the new policy to the old rollout policy.

The probability ratio is:

```

"""text
rho_t(theta)
= pi_theta(a_t | s_t) / pi_theta_old(a_t | s_t)
"""

```

Using log probabilities:

```

"""text
rho_t(theta)
= exp(log pi_theta(a_t | s_t) - log pi_theta_old(a_t | s_t))
"""

```

The clipped PPO objective is:

```

"""text
L_clip(theta)
= E[min(rho_t A_t,
clip(rho_t, 1-epsilon, 1+epsilon) A_t)]
"""

```

The minimized policy loss is:

```

"""text
L_policy = -L_clip
"""

```

The value loss is:

```

"""text
L_value = E[(V_theta(s_t) - R_t)^2]
"""

```

"""

The entropy bonus is:

"""text

$H = - \sum_a \pi_{\theta}(a | s_t) \log \pi_{\theta}(a | s_t)$

"""

The total PPO loss is:

"""text

$L_{\text{PPO}} = L_{\text{policy}} + c_v L_{\text{value}} - c_H H$

"""

where:

"""text

$c_v = \text{value_coef}$

$c_H = \text{entropy_coef}$

"""

In this implementation, the rollout receives one scalar reward, and that reward is broadcast to all generated action positions:

"""text

$R_t = R$ for all generated t

"""

This is a simple sequence-level PPO scaffold. A more advanced implementation could use token-aligned process rewards and reward-to-go.

14. GRPO

GRPO samples multiple completions for the same prompt.

For prompt ' q_i ', sample:

"""text

$a_{i,1}, a_{i,2}, \dots, a_{i,K}$

"""

Each completion receives reward:

"""text

$R_{i,k}$

"""

The group mean is:

```
"""text
mu_i = (1/K) sum_{k=1}^K R_{i,k}
"""
```

The group standard deviation is:

```
"""text
sigma_i = sqrt((1/K) sum_{k=1}^K (R_{i,k} - mu_i)^2)
"""
```

The group-relative advantage is:

```
"""text
A_{i,k} = (R_{i,k} - mu_i) / (sigma_i + epsilon)
"""
```

The clipped GRPO loss is PPO-like:

```
"""text
L_GRPO_policy
= -mean_{i,k,t} min(rho_{i,k,t} A_{i,k},
clip(rho_{i,k,t}, 1-epsilon, 1+epsilon) A_{i,k})
"""
```

GRPO also uses entropy regularization:

```
"""text
L_GRPO = L_GRPO_policy - c_H H
"""
```

There is no value loss in GRPO:

```
"""text
L_value is absent
"""
```

This is the key difference:

```
"""text
PPO baseline = learned value model
GRPO baseline = relative performance among samples for the same prompt
"""
```

15. Why Process Rewards Help

Terminal-only reward asks:

```
"""text
Was the final answer correct?
"""
```

Process reward asks:

```
"""text
Was the computation correct at each prefix?
"""
```

For parity, the process reward checks whether the model updates the parity bit after every input bit.

For modular counting, it checks the residue class after every input.

For 'a*b*', it checks the DFA state.

For 'a^n b^n', it checks phase and balance.

This is a much richer learning signal. It encourages the model to implement the recurrence:

```
"""text
s_t = f(s_{t-1}, x_t)
"""
```

instead of only learning a shortcut to the final label.

16. Length Generalization

The train set only contains lengths up to 40. A non-algorithmic model can exploit this. It may memorize short templates or learn length-specific heuristics.

Length generalization requires the learned transition rule to remain valid for longer sequences:

```
"""text
s_t = f(s_{t-1}, x_t)
"""
```

For parity:


```
"""text
f(p, x) = p XOR x
"""
```

For modular counting:

```
"""text
f(c, x) = (c + x) mod k
"""
```

For a^*b^* , f is the DFA transition function.

For $a^n b^n$, f updates phase and balance.

The central bet of the project is that rewarding the intermediate state makes this transition rule easier to learn.

17. Evaluation Metrics

The implementation reports three main metrics.

Process score:

```
"""text
mean R_process
"""
```

Terminal score:

```
"""text
mean R_terminal
"""
```

Exact score:

```
"""text
1[generated target prefix exactly equals oracle target]
"""
```

Exact score is strict. A model can have useful process behavior but fail exact match because of formatting mistakes. Terminal score is useful for language recognition, while process score is useful for diagnosing whether the model learned the intended computation.

18. Implementation Map

The major files are:

```
"""text
tasks.py      task generators, oracle traces, rewards
tokenizer.py  fixed symbolic tokenizer
model.py      causal Transformer policy and value model
train.py      SFT/PPO/GRPO training entrypoint
rl.py         rollout collection and PPO/GRPO updates
evaluate.py   length-bucket evaluation utilities
eval.py       checkpoint evaluation CLI
configs/      example experiment configs
"""
```

The implementation is intentionally compact so that experiments can be modified quickly.

19. Important Limitations

The current codebase is a research scaffold, not a full production RLHF implementation.

Important limitations:

- PPO broadcasts one sequence-level reward to all generated tokens.
- The process reward is computed after generation rather than inserted as per-token reward during rollout.
- PPO does not yet implement full reward-to-go or GAE.
- There is no KL penalty against a frozen reference model.
- Evaluation up to length 500 can be slow on CPU.
- The generation format is strict; malformed traces may receive poor reward even when partially meaningful.

These limitations are useful future improvement points.

20. Natural Next Extensions

A stronger PPO implementation would use token-aligned process reward:

```
"""text
r_t = oracle reward for the generated state token at time t
"""
```

Then reward-to-go would be:

```
"""text
G_t = sum_{u=t}^T gamma^{(u-t)} r_u
"""
```

GAE would use:

```
"""text
delta_t = r_t + gamma V(s_{t+1}) - V(s_t)
"""
```

and:

```
"""text
A_t^GAE = sum_l (gamma lambda)^l delta_{t+l}
"""
```

Other extensions:

- frozen reference policy and KL penalty
- deterministic decoding for evaluation
- curriculum schedules within 'n <= 40'
- per-length plots instead of only bucket averages
- multiple seeds and confidence intervals
- recurrent or stack-augmented baselines

21. How to Interpret Results

Strong evidence for algorithmic learning would be:

- high training process accuracy
- high training terminal accuracy
- high OOD process accuracy
- high OOD terminal accuracy
- graceful degradation as length increases
- process-reward PPO/GRPO outperforming terminal-only PPO/GRPO

A weak result would be:

```
"""text
high train terminal accuracy but poor OOD terminal accuracy
"""
```

That suggests memorization or short-length heuristics.

Another interesting result is:

““text
high process accuracy but low terminal accuracy
““

That may indicate formatting or final-token problems rather than a failure to learn the underlying state transition.

22. Summary

The project turns formal-language recognition into autoregressive trace generation. The Transformer policy generates symbolic computation traces. The oracle rewards either the final answer, the intermediate computation, or both.

PPO uses a learned value baseline and clipped policy updates. GRPO uses multiple completions for the same prompt and group-relative advantage normalization.

The central scientific claim being tested is:

““text
If the model learns the true state-transition rule, it should generalize
beyond the lengths it saw during training.
““

Process rewards are designed to make learning that rule easier.